

Computer Assignment 3

Min Zhu & Ziyi Wang

2025-10-22

```
load("proj_data.Rdata")
modell1 <- glm(Resultat ~ Alder + Kon + Utbildare,
              data = data_individ,
              family = "binomial")
summary(modell1)

##
## Call:
## glm(formula = Resultat ~ Alder + Kon + Utbildare, family = "binomial",
##      data = data_individ)
##
## Coefficients:
##              Estimate Std. Error z value Pr(>|z|)
## (Intercept)   -0.065131    0.253358  -0.257   0.79712
## Alder         -0.025122    0.008671  -2.897   0.00376 **
## KonMan         0.100567    0.137049   0.734   0.46307
## UtbildareTrafikskola 0.947498    0.158405   5.982 2.21e-09 ***
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## (Dispersion parameter for binomial family taken to be 1)
##
##      Null deviance: 1344.4  on 999  degrees of freedom
## Residual deviance: 1291.4  on 996  degrees of freedom
## AIC: 1299.4
##
## Number of Fisher Scoring iterations: 4
```

Task1

```
## Functions
## test1
# -----a-----
L <- function(theta,y,X){
  z <- as.vector(X %*% theta)
  p <- 1/(1+exp(-z))
  prod(dbinom(y, size=1, prob=p))
}

l <- function(theta,y,X){
  z <- as.vector(X %*% theta)
  # p <- plogis(z)
  p <- 1/(1+exp(-z))
  sum(dbinom(y, size=1, prob=p, log=TRUE))
}
```

```

}#using 'dbinom' in both L and L

S <- function(theta,y,X){
  z <- as.vector(X %**% theta)
  p <- 1/(1+exp(-z))
  t(X) %**% (y-p)
}

I <- function(theta,y,X){
  z <- as.vector(X %**% theta)
  p <- 1/(1+exp(-z))
  D <- diag(as.vector(p*(1-p)))
  t(X) %**% D %**% X
}

# -----b-----
NR <- function(theta0, niter, y, X){
  for (i in 1:niter) {
    score <- S(theta0, y, X)
    fisher <- I(theta0, y, X)
    theta0 <- theta0 + solve(fisher, score)
  }
  return(theta0)
}

# coefficient of glm method
theta_mo <- coef(model1)
theta_mo <- as.numeric(theta_mo)

y <- matrix(data_individ$Resultat, ncol = 1)
X <- model.matrix(Resultat ~ Alder + Kon + Utbildare,
                  data = data_individ)

## Using functions from Part 1
theta0 <- c(0,0,0,0)
max_iter <- 100
for (i in 1:max_iter){
  theta0 <- NR(theta0, niter=1, y, X)
  theta_hat <- theta0
  if (all(round(theta_hat, 2) == round(theta_mo, 2))) {
    break
  }
}

## Computing AIC
p <- 4
log_Likelihood <- l(theta_hat,y,X)
AIC_f <- -2* log_Likelihood + 2*p
cat("Value of AIC using previous funcitons:",AIC_f,"\n")

```

```
## Value of AIC using previous functions: 1299.382
```

```
cat("AIC value of model1:", AIC(model1), "\n")
```

```
## AIC value of model1: 1299.382
```

The AIC value computed using the previous functions agrees with the value obtained from R.

```
## Leave-one-out Cross validation
```

```
n <- nrow(X)
```

```
log_li_cv <- numeric(n)
```

```
tolerance <- 1e-4
```

```
max_iter <- 100
```

```
for (i in 1:n){
```

```
  # split sample: training
```

```
  X_l <- X[-i,, drop = FALSE]
```

```
  y_l <- y[-i]
```

```
  theta_hat <- c(0, 0, 0, 0)
```

```
  for (j in 1:max_iter){
```

```
    theta_prev <- theta_hat
```

```
    theta_hat <- NR(theta_prev, niter=1, y_l, X_l)
```

```
    if (all(abs(theta_hat - theta_prev) < tolerance)) {  
      break  
    }
```

```
  }  
  log_li_cv[i] <- l(theta_hat, y[i], X[i,, drop = FALSE])  
}
```

```
log_final <- mean(log_li_cv)
```

```
cat("cross-validated average log-likelihood:", log_final, "\n")
```

```
## cross-validated average log-likelihood: -0.6497103
```

```
AIC_cv <- -2*sum(log_li_cv)
```

```
cat("AIC computing from cross-validated method:", AIC_cv, "\n")
```

```
## AIC computing from cross-validated method: 1299.421
```

The result of cross-validation is very close to the previous value of AIC.

Task 2

```
## Define post function
```

```
post <- function(theta, y, X) {
```

```
  eta <- X %*% theta
```

```
  p <- 1 / (1 + exp(-eta))
```

```
  likelihood <- prod(p^y * (1 - p)^(1 - y))  
}
```

```

# vague prior:  $\theta \sim N(0, 100I)$ 
prior <- prod(dnorm(theta, mean = 0, sd = 10))

# Posterior
return(likelihood * prior)
}

Xtest <- cbind(1, 18:25, rep(c(0,1),4), rep(c(1,1,0,0),2))
ytest <- c(rep(TRUE, 4), rep(FALSE, 4))
post(c(260, -10, 10, -20), ytest, Xtest) / post(c(270, -15, 15, -25), ytest,
Xtest)

## [1] 3.707556e+25

```

The computed value is aligned with the answer given.

Task 3

```

theta_mle <- coef(model1) # initial values
sigma_step <- summary(model1)$coefficients[, "Std. Error"] # step sizes

# Define log-posterior function
log_posterior <- function(theta, X, y) {
  eta <- X %*% theta
  p <- 1 / (1 + exp(-eta))
  log_lik <- sum(y * log(p) + (1 - y) * log(1 - p))

  # Prior:  $\theta \sim N(0, 100I)$  - vague prior
  log_prior <- sum(dnorm(theta, mean = 0, sd = 10, log = TRUE))

  # Log-posterior = Log-likelihood + Log-prior
  return(log_lik + log_prior)
}

# Data for MCMC
X <- model.matrix(Resultat ~ Alder + Kon + Utbildare, data = data_individ)
y <- data_individ$Resultat

# Metropolis-Hastings Algorithm
N <- 10000 # Number of iterations
n_params <- length(theta_mle)
theta_samples <- matrix(nrow = N, ncol = n_params)
colnames(theta_samples) <- names(theta_mle)

# Initialize with MLE estimates
theta_samples[1, ] <- theta_mle

# Acceptance counter

```

```

count <- 0

# Main MCMC Loop
set.seed(021120)
for (i in 2:N) {
  # Generate candidate from proposal distribution:  $N(\theta_{i-1}, \sigma_{step}^2)$ 
  theta_star <- theta_samples[i-1, ] + rnorm(n_params) * sigma_step

  # Calculate acceptance probability (log scale)
  log_alpha <- log_posterior(theta_star, X, y) -
log_posterior(theta_samples[i-1, ], X, y)

  # Accept or reject candidate
  if (log(runif(1)) < log_alpha) {
    theta_samples[i, ] <- theta_star
    count <- count + 1
  } else {
    theta_samples[i, ] <- theta_samples[i-1, ]
  }
}

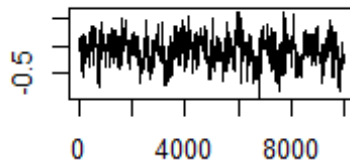
# Calculate acceptance rate
accept_rate <- count / (N - 1)
cat("Acceptance rate:", accept_rate, "\n")

## Acceptance rate: 0.1391139

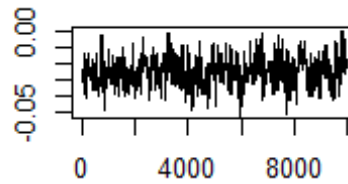
# Plot parameter traces
par(mfrow = c(2, 2))
for (j in 1:n_params) {
  plot(theta_samples[, j], type = "l",
       main = paste("Parameter Trace:", colnames(theta_samples)[j]),
       xlab = "", ylab = "")
}

```

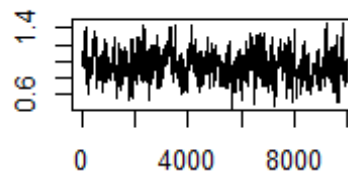
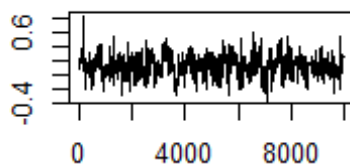
Parameter Trace: (Intercept)



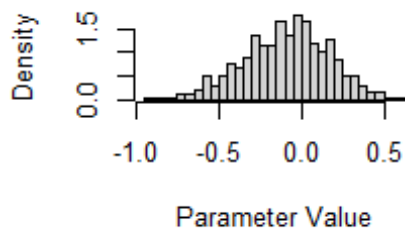
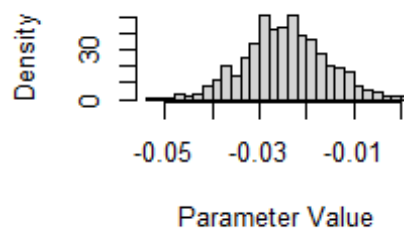
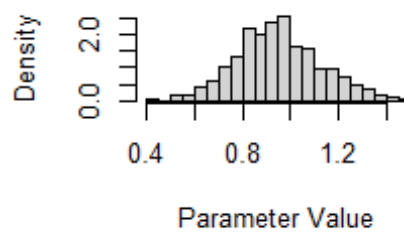
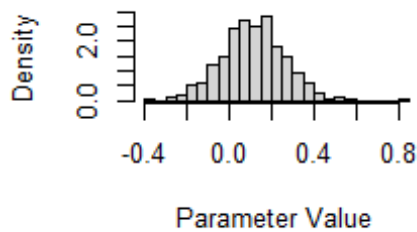
Parameter Trace: Alder



Parameter Trace: KonMan 'arameter Trace: Utbildare Trafik:



```
# Plot posterior distributions
par(mfrow = c(2, 2))
for (j in 1:n_params) {
  hist(theta_samples[, j], breaks = 30,
       main = paste("Posterior Distribution:", colnames(theta_samples)[j]),
       xlab = "Parameter Value", probability = TRUE)
}
```

Posterior Distribution: (Intercept)**Posterior Distribution: Alder****Posterior Distribution: KonMan**

```
# Calculate posterior statistics
```

```
posterior_means <- apply(theta_samples, 2, mean)
posterior_quantiles <- apply(theta_samples, 2, quantile, probs = c(0.025,
0.975))
```

```
# Create comparison table
```

```
results_comparison <- data.frame(
  Parameter = names(theta_mle),
  MLE = theta_mle,
  Bayesian_Mean = posterior_means,
  Bayesian_CI_Lower = posterior_quantiles[1, ],
  Bayesian_CI_Upper = posterior_quantiles[2, ],
  MLE_SE = sigma_step
)
```

```
print(results_comparison)
```

	Parameter	MLE	Bayesian_Mean
## (Intercept)	(Intercept)	-0.06513133	-0.08912064
## Alder	Alder	-0.02512155	-0.02448030
## KonMan	KonMan	0.10056720	0.11267987
## UtbildareTrafikskola	UtbildareTrafikskola	0.94749776	0.94869543
	Bayesian_CI_Lower	Bayesian_CI_Upper	MLE_SE
## (Intercept)	-0.59836805	0.362247382	0.253357834
## Alder	-0.04130372	-0.006465947	0.008670842
## KonMan	-0.17743225	0.395700501	0.137048678
## UtbildareTrafikskola	0.62628137	1.307897728	0.158404569

```

# Predict probability for new observation
# My age 22, female, private education
x_star <- c(1, 22, 0, 0) # Intercept, Age, female, Private education

# Calculate predicted probability
pred_probs <- apply(theta_samples, 1, function(theta) {
  eta <- sum(x_star * theta)
  return(1 / (1 + exp(-eta)))
})

# Calculate posterior predictive probability
posterior_pred_prob <- mean(pred_probs)

cat("Posterior predictive probability of passing test:",
    round(posterior_pred_prob, 4), "\n")

## Posterior predictive probability of passing test: 0.3486

# Plot posterior distribution of predictive probability
hist(pred_probs, breaks = 30, main = "Posterior Predictive Probability
Distribution",
     xlab = "P(Y* = 1 | y)", probability = TRUE)
abline(v = posterior_pred_prob, col = "red", lwd = 2, lty = 2)
legend("topright", legend = paste("Mean =", round(posterior_pred_prob, 4)),
     col = "red", lty = 2)

```

